

# Лабораторна робота №3

## з теми «Написання модульних тестів»

Виконала:

ст. гр. ІП-22-1

Чемна (Микитин) О. І.

Перевірив:

Храбатин Р. І.

**Мета:** Навчитися створювати модульні тести, розібратися в існуючих інструментах для створення модульних тестів та навчитися їх обирати.

### **Короткі теоретичні відомості:**

Модульне тестування використовується для перевірки невеликих частин програмного коду, наприклад окремих функцій.

Такі тести дозволяють швидко перевірити, чи правильно працює конкретний фрагмент програми без запуску всієї системи. Завдяки цьому можна швидше знаходити помилки та впевненіше змінювати код у майбутньому.

Для написання модульних тестів у JavaScript часто використовують спеціальні інструменти запуску тестів і бібліотеки перевірок. До популярних інструментів належать Jest, Mocha, Karma, Jasmine та вбудований Node.js test-runner.

У цій роботі використовується Jest, оскільки він є простим у налаштуванні, має зручний синтаксис і дозволяє швидко писати перевірки через конструкції `describe`, `test` та `expect`.

### **Завдання:**

#### **Мій варіант — 30.**

##### **Завдання перше:**

Згідно із вказівками вище налаштувати середовище для написання юніт-тестів.

##### **Завдання друге:**

Взяти один варіант з таблиці нижче і написати по п'ять тестів до кожної функції. Якщо Ви маєте бажання написати власну функцію – це тільки вітається, особливо коли в неї є більш-менш складна логіка яку можна протестувати кількома способами. Для легшого розуміння раджу ознайомитися з документацією за посиланням: <https://jestjs.io/docs/expect>

##### **Завдання третє:**

Завантажити код на власний/навчальний репозиторій GitHub, або інший схожий ресурс та поділитися посиланням.

Список функцій для тестування:

|    |                                                                                                                                                                                                                             |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 30 | <pre> export  const  setFullYear  =  (date,  year)  =&gt; date.setFullYear(year) export  const  setMonth    =  (date,  month)  =&gt; date.setMonth(month) export const setDate = (date, day) =&gt; date.setDate(day) </pre> |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Виконання:

Спочатку я створила окремий каталог для виконання лабораторної роботи:

```

oksanka@debian:~/Documents/automated_testing$ mkdir unit-testing
oksanka@debian:~/Documents/automated_testing$ cd unit-testing

```

Після цього я ініціалізувала Node.js-проект за допомогою команди:

```

oksanka@debian:~/Documents/automated_testing/unit-testing$ npm init -y
Wrote to /home/oksanka/Documents/automated_testing/unit-testing/package.json:

{
  "name": "unit-testing",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

oksanka@debian:~/Documents/automated_testing/unit-testing$

```

Далі я встановила Jest як залежність для розробки:

```

oksanka@debian:~/Documents/automated_testing/unit-testing$ npm install --save-dev jest
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me

added 293 packages, and audited 294 packages in 32s

44 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
oksanka@debian:~/Documents/automated_testing/unit-testing$

```

Оскільки я використовувала синтаксис `export` та `import`, я також встановила Babel-залежності для коректної роботи модулів у Jest:

```
oksanka@debian:~/Documents/automated_testing/unit-testing$ npm install --save-dev @babel/core @babel/preset-env babel-jest

added 95 packages, and audited 389 packages in 8s

49 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
oksanka@debian:~/Documents/automated_testing/unit-testing$
```

Після цього я створила файл `.babelrc`. У файл `.babelrc` я додала такі налаштування:

```
.babelrc > ...
1 {
2   "presets": ["@babel/preset-env"]
3 }
4
```

Далі я відкрила файл `package.json` і змінила секцію `scripts`, щоб тести можна було запускати командою `npm test`:

```
> Debug
6 "scripts": {
7   "test": "jest"
8 },
9
```

Після налаштування середовища я створила файл `dateFunctions.js`, у якому розмістила функції з мого варіанту:

```
dateFunctions.js > ...
1 export const setFullYear = (date, year) => date.setFullYear(year);
2
3 export const setMonth = (date, month) => date.setMonth(month);
4
5 export const setDate = (date, day) => date.setDate(day);
6
```

Після цього я створила файл з тестами `dateFunctions.test.js`. У файл `dateFunctions.test.js` я імпортувала функції, які потрібно було протестувати:

```
dateFunctions.test.js
1 import { setFullYear, setMonth, setDate } from "../dateFunctions";
2
```

Далі я написала модульні тести для функції `setFullYear`. Ця функція встановлює рік для об'єкта `Date`:

```

2
3 describe("setFullYear", () => {
4     test("should set year to 2025", () => {
5         const date = new Date(2020, 0, 1);
6
7         setFullYear(date, 2025);
8
9         expect(date.getFullYear()).toBe(2025);
10    });
11
12    test("should set year to 2000", () => {
13        const date = new Date(2024, 5, 15);
14
15        setFullYear(date, 2000);
16
17        expect(date.getFullYear()).toBe(2000);
18    });
19
20    test("should keep month unchanged after setting year", () => {
21        const date = new Date(2020, 7, 10);
22
23        setFullYear(date, 2030);
24
25        expect(date.getMonth()).toBe(7);
26    });
27
28    test("should keep day unchanged after setting year", () => {
29        const date = new Date(2020, 3, 22);
30
31        setFullYear(date, 2022);
32
33        expect(date.getDate()).toBe(22);
34    });
35
36    test("should return timestamp after setting year", () => {
37        const date = new Date(2020, 0, 1);
38
39        const result = setFullYear(date, 2021);
40
41        expect(typeof result).toBe("number");
42    });
43 });

```

Після цього я написала модульні тести для функції `setMonth`. Ця функція встановлює місяць для об'єкта `Date`. Я врахувала, що в JavaScript місяці рахуються з нуля, тобто січень — це 0, а грудень — це 11:

```

44
45 describe("setMonth", () => {
46     test("should set month to January", () => {
47         const date = new Date(2024, 5, 10);
48
49         setMonth(date, 0);
50
51         expect(date.getMonth()).toBe(0);
52     });
53
54     test("should set month to December", () => {
55         const date = new Date(2024, 2, 10);
56
57         setMonth(date, 11);
58
59         expect(date.getMonth()).toBe(11);
60     });
61
62     test("should set month to June", () => {
63         const date = new Date(2024, 0, 10);
64
65         setMonth(date, 5);
66
67         expect(date.getMonth()).toBe(5);
68     });
69
70     test("should keep year unchanged after setting month", () => {
71         const date = new Date(2024, 1, 15);
72
73         setMonth(date, 8);
74
75         expect(date.getFullYear()).toBe(2024);
76     });
77
78     test("should return timestamp after setting month", () => {
79         const date = new Date(2024, 1, 15);
80
81         const result = setMonth(date, 3);
82
83         expect(typeof result).toBe("number");
84     });
85 });

```

Далі я написала модульні тести для функції `setDate`. Ця функція встановлює день місяця для об'єкта `Date`:

```

87 describe("setDate", () => {
88     test("should set day to 1", () => {
89         const date = new Date(2024, 0, 15);
90
91         setDate(date, 1);
92
93         expect(date.getDate()).toBe(1);
94     });
95
96     test("should set day to 10", () => {
97         const date = new Date(2024, 0, 1);
98
99         setDate(date, 10);
100
101         expect(date.getDate()).toBe(10);
102     });
103
104     test("should set day to 28", () => {
105         const date = new Date(2024, 1, 1);
106
107         setDate(date, 28);
108
109         expect(date.getDate()).toBe(28);
110     });
111
112     test("should keep month unchanged when day is valid", () => {
113         const date = new Date(2024, 6, 1);
114
115         setDate(date, 20);
116
117         expect(date.getMonth()).toBe(6);
118     });
119
120     test("should return timestamp after setting day", () => {
121         const date = new Date(2024, 0, 1);
122
123         const result = setDate(date, 15);
124
125         expect(typeof result).toBe("number");
126     });
127 });

```

Після написання тестів я запустила їх у терміналі командою:

```

● oksanka@debian:~/Documents/automated_testing/unit-testing$ npm test

> unit-testing@1.0.0 test
> jest

PASS ./dateFunctions.test.js
  setFullYear
    ✓ should set year to 2025 (1 ms)
    ✓ should set year to 2000
    ✓ should keep month unchanged after setting year
    ✓ should keep day unchanged after setting year
    ✓ should return timestamp after setting year (1 ms)
  setMonth
    ✓ should set month to January
    ✓ should set month to December
    ✓ should set month to June
    ✓ should keep year unchanged after setting month
    ✓ should return timestamp after setting month (1 ms)
  setDate
    ✓ should set day to 1 (1 ms)
    ✓ should set day to 10
    ✓ should set day to 28
    ✓ should keep month unchanged when day is valid (4 ms)
    ✓ should return timestamp after setting day

Test Suites: 1 passed, 1 total
Tests:       15 passed, 15 total
Snapshots:   0 total
Time:        0.374 s
Ran all test suites.
● oksanka@debian:~/Documents/automated_testing/unit-testing$

```

У результаті запуску Jest виконав усі створені тести. Загалом було написано 15 модульних тестів, тобто по 5 тестів для кожної з трьох функцій мого варіанту.

Після успішного запуску тестів я підготувала проєкт до завантаження на GitHub. У мене раніше уже був ініціалізований Git-репозиторій для виконання цих робіт.

Тому першим ділом я перевірила стан файлів:

```

oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Невідстежувані файли:
  (скористайтесь "git add <файл>...", щоб додати до майбутнього коміту)
    unit-testing/

нічого не додано до коміту, але є невідстежувані файли (скористайтесь "git add" для відстежування)
oksanka@debian:~/Documents/automated_testing$

```

Далі я додала всі файли до індексації:



```

oksanka@debian:~/Documents/automated_testing$ git add .
oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Зміни, додані до майбутнього коміту:
(використовуйте "git restore --staged <файл>...", щоб видалити з індексу)
    новий файл:    unit-testing/.babelrc
    новий файл:    unit-testing/.gitignore
    новий файл:    unit-testing/dateFunctions.js
    новий файл:    unit-testing/dateFunctions.test.js
    новий файл:    unit-testing/package-lock.json
    новий файл:    unit-testing/package.json

oksanka@debian:~/Documents/automated_testing$ |

```

Після цього я створила коміт:

```

oksanka@debian:~/Documents/automated_testing$ git commit -m "Add unit tests for lab 3"
[main 7ed5b27] Add unit tests for lab 3
6 files changed, 6038 insertions(+)
create mode 100644 unit-testing/.babelrc
create mode 100644 unit-testing/.gitignore
create mode 100644 unit-testing/dateFunctions.js
create mode 100644 unit-testing/dateFunctions.test.js
create mode 100644 unit-testing/package-lock.json
create mode 100644 unit-testing/package.json
oksanka@debian:~/Documents/automated_testing$

```

Потім я завантажила проєкт на GitHub:

```

oksanka@debian:~/Documents/automated_testing$ git push -u origin main
Username for 'https://github.com': OksanaMykytyn
Password for 'https://OksanaMykytyn@github.com':
Перерахування об'єктів: 100%, готово.
Підрахунок об'єктів: 100% (10/10), готово.
Дельта компресія з використанням до 4 потоків
Компресія об'єктів: 100% (7/7), готово.
Запис об'єктів: 100% (9/9), 46.57 KiB | 9.31 MiB/c, готово.
Всього 9 (дельта 1), повторно використано 0 (дельта 0), повторно використано пакуноків 0
(з 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/OksanaMykytyn/automated_testing.git
  24d8d93..7ed5b27  main -> main
гілку "main" налаштовано на відстежування "origin/main".
oksanka@debian:~/Documents/automated_testing$

```

**Посилання на GitHub:**

[https://github.com/OksanaMykytyn/automated\\_testing](https://github.com/OksanaMykytyn/automated_testing)

**Висновок:** під час лабораторної роботи я навчилася створювати модульні тести для JavaScript-функцій за допомогою Jest. Я налаштувала Node.js-проєкт, встановила Jest і додала Babel для підтримки синтаксису import та export. Для свого варіанту 30 я реалізувала три функції для роботи з датою: setFullYear, setMonth та setDate. До кожної функції я написала по п'ять тестів, тобто загалом створила 15 модульних тестів. Виконану роботу надіслала у віддалений репозиторій.